

Лекция 8

Работа с файлами в языке Си

1 Создание файловой переменной

Файл – именованная область памяти, выделенная для хранения данных

Формат объявления указателя на файл :

```
FILE *логическое_имя_файла;
```

Примеры:

```
#include <stdio.h>
```

```
FILE *f1 = NULL;
```

```
FILE *f1, *f2;
```

2 Открытие и закрытие файла

Открытие файла:

```
FILE *fopen(const char *filename, const char *mode);
```

filename – физический путь расположения файла

mode – режим открытия

Функция **fopen** возвращает указатель на файл, если тот был успешно открыт. В противном случае – NULL.

Примеры написания пути файла:

"data.txt"

"..\\files\\data.txt"

"d:\\temp\\data.txt"

Режим	Описание
r	Чтение из файла. Файл должен существовать.
w	Создание файла для запись данных. Если файл с таким именем уже существует, то его содержимое будет потеряно.
a	Запись в конец файла. Если файл не существует, то будет создан новый файл.
r+	Чтение и запись данных. Файл должен существовать.
w+	Чтение и запись данных. Создаётся новый файл. Если файл с таким именем уже существует, то его содержимое будет потеряно.
a+	Запись в конец файла и чтение данных. Если файл не существует, то будет создан новый файл.

Если необходимо открыть файл в бинарном режиме, то добавляется буква b, например “rb”, “wb”, “ab”, или, для смешанного режима “ab+”, “wb+”, “ab+”. Вместо b можно добавлять букву t, тогда файл будет открываться в текстовом режиме.

Заккрытие файла:

```
int fclose(FILE *stream);
```

stream - указатель на
открытый файл

Функция **fclose** возвращает:
0 – файл успешно закрыт;
EOF (-1) – произошла ошибка
закрытия файла.

*Создание файла «file.txt» для
записи данных:*

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w");
7      fclose(f);
8      return 0;
9  }
```

*Открытие файла «file.txt» для
записи данных в конец (файл
должен существовать) :*

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "a");
7      fclose(f);
8      return 0;
9  }
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w+");
7
8      if (fclose(f)==0) printf("Файл успешно закрыт!");
9      else printf("Ошибка!");
10
11     return 0;
12 }

```

Файл успешно закрыт!

```

1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "");
7      if (f == NULL) printf("Файл не открылся!\n");
8
9      if (fclose(f)==EOF) printf("Ошибка!");
10     else printf("Файл успешно закрыт!");
11
12     return 0;
13 }

```

Файл не открылся!
Ошибка!

3 Изменение режима доступа к файлу

FILE* freopen (char * filename, char * mode, FILE *p);

filename – физический путь расположения файла

mode – новый режим открытия файла

p – указатель на файл

Действия функции ***freopen***:

- 1) закрытие файла, заданного в третьем параметре p;
- 2) открытие файла filename с заданным режимом доступа mode.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w");
7
8      f = freopen ("file.txt", "r", f);
9
10     fclose(f);
11     return 0;
12 }
```

4 Временные файлы

Создание на диске временного файла с правами доступа "w+b":

FILE tmpfile (void);*

После завершения работы программы или закрытия временного файла он автоматически удаляется

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = tmpfile();
7
8      if (f != NULL) printf("Создан временный файл!");
9      else printf("Не создан временный файл!");
10
11     return 0;
12 }
```

Создан временный файл!

5 Навигация по файлу

Проверка на достижение конца файла:

```
int feof(FILE *f);
```

Функция **feof** возвращает:

- **0** – если конец файла еще не достигнут.
- **!0** – достигнут конец файла.

Изменение текущего смещения в файле:

```
int fseek(FILE *f, long size, int code);
```

Функция **fseek** возвращает:

- **0** – все нормально,
- **!0** – произошла ошибка.

Значение параметра **size** задает количество байт, на которое необходимо сместить указатель в файле **f**, в направлении параметра **code**, который может принимать следующие значения: **SEEK_SET** (смещение от начала файла); **SEEK_CUR** (смещение от текущей позиции); **SEEK_END** (смещение от конца файла).

6 Запись данных в текстовый файл

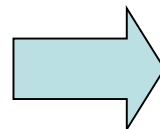
Запись символа:

```
int fputc(int c, FILE *stream);
```

Функция **fputc** возвращает:

- **код записанного символа** – все нормально,
- **EOF** – произошла ошибка.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w");
7
8      fputc('A', f);
9      fputc('.', f);
10     fputc('\n', f);
11     fputc(66, f);
12
13     fclose(f);
14     return 0;
15 }
```



file – Блокнот

Файл Правка Формат

A.

B

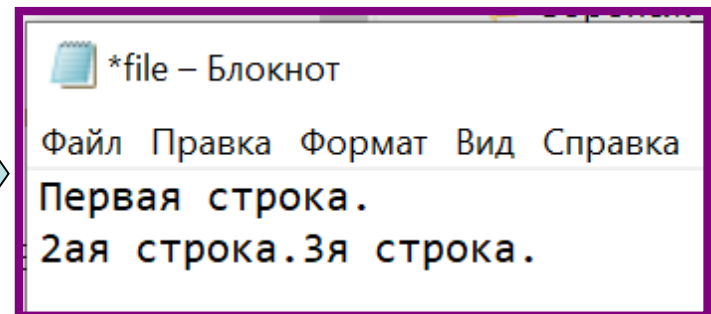
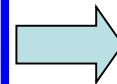
Запись строки:

```
int fputs(const char *string, FILE *stream);
```

Функция **fputs** возвращает:

- **число записанных символов** – все нормально,
- **EOF** – произошла ошибка.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "wt");
7
8      fputs("Первая строка.\n", f);
9      fputs("2ая строка.", f);
10     fputs("3я строка.", f);
11
12     fclose(f);
13     return 0;
14 }
```



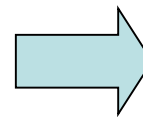
Запись строки:

```
int fprintf(FILE *stream, const char *format, [arg] ...);
```

Функция **fprintf** возвращает:

- **число записанных символов** – если все нормально,
- **отрицательное значение** – если ошибка.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w");
7
8      int a = 1;
9      float b = 2.2, c = 3.2;
10
11     fprintf(f, "a = %d\n", a);
12     fprintf(f, "b = %f, c = %f", b, c);
13
14     fclose(f);
15     return 0;
16 }
```



 file – Блокнот

Файл Правка Формат Вид Справка

a = 1
b = 2.200000, c = 3.200000

7 Запись данных в файл в бинарном режиме

unsigned **fwrite** (void **p*, unsigned *size*, unsigned *n*, FILE **f*);

В файл *f* будет записано *n* блоков размером *size* байт каждый.
Записываемые данные начинаются с адреса *p*

```
1  #include <stdio.h>
2
3  struct people
4  {
5      char name[20];
6      int age;
7  };
8
9  int main()
10 {
11     struct people P[3]={ {"Anna", 10},
12                           {"Bob", 20},
13                           {"Alex", 30} };
14
15     FILE *f = NULL;
16     f = fopen("file.bin", "wb");
17
18     fwrite (P, sizeof(struct people), 3, f);
19
20     fclose(f);
21     return 0;
22 }
```

8 Чтение данных из текстового файла

Чтение символа:

```
int fgetc(FILE *stream);
```

Функция **fgetc** возвращает:

- **код символа** — если все нормально,
- **EOF** — если ошибка или достигнут конец файла.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "wt");
7
8      fputc('A', f);
9      fputc('.', f);
10     fputc('\n', f);
11     fputc(66, f);
12
13     f = freopen("file.txt", "rt", f);
14
15     char c = fgetc(f);
16     do {
17         printf("%c", c);
18         c = fgetc(f);
19     }
20     while (!feof(f));
21
22     fclose(f);
23     return 0;
24 }
```

A.
B

Чтение строки:

```
char * fgets(char * buffer, int maxlen, FILE *stream);
```

Функция **fgets** возвращает:

- **buffer** – все нормально,
- **NULL** – ошибка или достигнут конец файла.

СТРОКА 1: Первая строка.
СТРОКА 2: 2ая строка.3я строка.



```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w+");
7
8      fputs("Первая строка.\n", f);
9      fputs("2ая строка.", f);
10     fputs("3я строка.", f);
11
12     char s[30];
13
14     fseek(f,0,SEEK_SET);
15
16     int i=0;
17     do {
18         printf("СТРОКА %d: ", ++i);
19         fgets(s, 30, f);
20         printf("%s", s);
21     }
22     while (!feof(f));
23
24     fclose(f);
25     return 0;
26 }
```

Чтение строки:

```
int fscanf(FILE *stream, const char * format, [arg] ...);
```

Функция **fscanf** возвращает:

- **>0** – число успешно прочитанных переменных,
- **0** – ни одна из переменных не была успешно прочитана,
- **EOF** – ошибка или достигнут конец файла.



file – Блокнот

Файл Правка Формат Вид Справка

a= 1 b= 2.200000

a= 1 b= 2.200000

```
1  #include <stdio.h>
2
3  int main()
4  {
5      FILE *f = NULL;
6      f = fopen("file.txt", "w+");
7
8      int a = 1;
9      float b = 2.2;
10     fprintf(f, "a= %d b= %f", a, b);
11
12     fseek(f,0,SEEK_SET);
13
14     char s1[5], s2[5];
15     int c;
16     float d;
17
18     fscanf(f, "%s %d %s %f", s1, &c, s2, &d);
19     printf("%s %d %s %f", s1, c, s2, d);
20
21     fclose(f);
22     return 0;
23 }
```


9 Чтение данных из файла в бинарном режиме

unsigned fread (void ****p***, unsigned ***size***, unsigned ***n***, FILE ****f***);

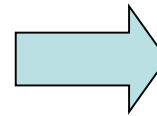
Функция ***fread*** выполняет считывание из файла ***f*** ***n*** блоков размером ***size*** байт каждый в область памяти. Данные записываются по адресу ***p***.

В случае успеха функция возвращает количество считанных блоков. При возникновении ошибки или по достижении признака окончания файла – значение ***EOF***.

```

1  #include <stdio.h>
2
3  struct people
4  {
5      char name[20];
6      int age;
7  };
8
9  int main()
10 {
11     system("chcp 1251");
12     system("cls");
13
14     struct people P[3]={ {"Anna", 10},
15                           {"Bob", 20},
16                           {"Alex", 30} };
17     FILE *f = NULL;
18     f = fopen("file.bin", "w+b");
19
20     fwrite(P, sizeof(struct people), 3, f);
21
22     fseek(f,0,SEEK_SET);
23     struct people P1[2];
24
25     fread(P1, sizeof(struct people), 2, f);
26
27     printf("%s %d\n", P1[0].name, P1[0].age);
28     printf("%s %d\n", P1[1].name, P1[1].age);
29
30     fclose(f);
31     return 0;
32 }

```



```

Anna 10
Bob 20

```

10 Удаление и переименование файлов

Функция удаления файла:

```
int remove(const char *filename);
```

Функция переименования файла:

```
int rename(const char *fname, const char *nname);
```

Функции **remove** и **rename** возвращают:

- **0** — в случае успеха,
- **!0** — в противном случае.